

A Statistical Model of Privacy-Budget Consumption in Repeated Aggregate SQL Workloads

CMP653 Term Project — Final Paper

Refik Can Öztaş
Hacettepe University
Ankara, Turkey
N25142279

Abstract

Differentially private (DP) SQL systems answer aggregate queries by adding calibrated noise, but they treat each query in isolation and charge a fresh slice of a finite privacy budget for every release. Real analytical workloads, however, *repeat*: dashboards refresh the same tiles, and drill-downs sweep the same templates. We present a closed-form analytical model that relates a workload’s template-repetition structure to its DP privacy-budget consumption. The model’s core is the expected distinct-template count $\mathbb{E}[u_k] = \sum_i [1 - (1 - p_i)^k]$; from it we derive the workload-aware budget $\varepsilon_q \mathbb{E}[u_k]$, a savings law $S(k) = 1 - \mathbb{E}[u_k]/k$ with two limits (deterministic repetition recovers the $1 - 1/k$ rule; uniform-over-many recovers naive composition), a Zipf-monotonicity result, a variance-aware concentration bound, and a utility ratio $k/\mathbb{E}[u_k]$. We instantiate the model in a unit-tested DP-SQL middleware over DuckDB/PostgreSQL that supports COUNT/SUM/AVG/GROUP BY and top- k (as DP post-processing), and validate it on a 4,155-trial campaign over Zipf workloads, TPC-H (SF=1 and SF=10), and UCI Adult: the model predicts $\mathbb{E}[u_k]$ within $< 3\%$ in 21/24 main-grid cells, is invariant to data scale and to ε_q , and holds *exactly* on a constructed mixed workload. We add two leakage experiments (membership inference and a differencing reconstruction) that confirm the per-query calibration and expose the looseness of cumulative-budget bounds. We are explicit about limits: the validation is a Monte-Carlo self-consistency check, not a real-trace study, and savings vanish on genuinely unique workloads—which the model predicts rather than hides.

1 Introduction and Motivation

Why this matters. Aggregate statistics leak. A pair of COUNT queries that differ by one predicate reveals an individual’s attribute by differencing; repeated over a dataset, such queries reconstruct it [3]. Differential privacy (DP) [1, 2] is the standard defense: perturb each answer with noise calibrated to the query’s sensitivity and a per-query budget ε_q , and bound the cumulative privacy loss by composition. The budget is a hard, non-renewable resource: once the total $\sum \varepsilon_q$ reaches a cap B , no further query can be answered without breaking the guarantee. A data custodian therefore needs to understand, and ideally predict, *how fast a workload spends its budget*.

A concrete pain. Consider an analytics team that publishes a privacy-protected dashboard: a fixed set of tiles (counts, sums, group-by breakdowns) refreshed on a schedule, plus occasional ad-hoc drill-downs. The custodian sets a monthly budget B and a per-query ε_q . Two failures follow from treating queries in isolation.

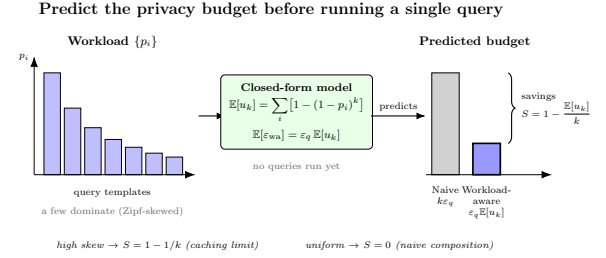


Figure 1: Naive composition charges every one of k queries; workload-aware accounting charges only the u_k distinct templates, since repeats are served from cache by post-processing. The model predicts $\mathbb{E}[u_k]$, and hence the budget, before execution.

If ε_q is set to meet an accuracy target, naive composition exhausts B after only B/ε_q releases and the dashboard goes dark mid-month. If ε_q is set small enough to survive the month, every tile is needlessly noisy. The custodian cannot tell, in advance, which will happen—because no model relates the *shape* of the workload (how much it repeats) to the budget it will spend. The repetition is exactly the structure our model exploits: the same dashboard tiles recur, so the budget is governed by the handful of *distinct* templates, not the hundreds of refreshes.

The opening: workloads repeat. Existing DP-SQL systems (PINQ [4], PrivateSQL [5], Chorus [6]) charge every query a fresh ε_q and account for them in isolation. Yet analytical workloads are highly repetitive: a monitoring dashboard re-issues the same KPI tile on a schedule, and an analyst’s drill-down revisits the same templates with different literals. Under exact-repeat caching, a repeated answer is free (it is *post-processing* of an already-released value), so the budget a workload of length k consumes is governed not by k but by its number of *distinct* queries u_k (Figure 1). The central question of this project is whether u_k —and hence the budget—can be *predicted from the workload’s repetition structure*, in closed form, before the workload runs.

Contributions.

- (1) **An analytical model** (Section 3) that, given a template distribution $\{p_i\}$ and length k , predicts the expected distinct-query count $\mathbb{E}[u_k]$, the workload-aware budget $\varepsilon_q \mathbb{E}[u_k]$, and the savings ratio $S(k)$, in closed form. Five propositions cover $\mathbb{E}[u_k]$, savings, the Zipf-parameterized case,

concentration, and per-query utility, with two named limits that recover the folklore $1 - 1/k$ rule and naive composition as corners.

- (2) **A middleware** that instantiates the model over DuckDB/PostgreSQL with four execution modes, exact-repeat caching, a temporal (staleness/update) extension, and **top- k as DP post-processing**.
- (3) **Empirical validation** (Section 4) on a 4,155-trial campaign: the model predicts $\mathbb{E}[u_k]$ within $< 3\%$ in 21/24 main-grid cells, is invariant to data scale (SF=1 vs SF=10) and to ε_q , and is exact on a constructed mixed workload; plus two leakage experiments and a runtime analysis.

Throughout, we aim to be balanced and explicit about scope: the headline validation is a self-consistency check (Section 6), and we report the regimes where the model predicts *no* benefit rather than overclaiming. The predictive allocator that the model also enables, and the learned-allocator comparison, are summarized in Appendix A; the negative result on semantic caching is in Appendix B.

2 Related Work

DP-SQL systems. PINQ [4] introduced composable DP query operators; PrivateSQL [5] allocates budget across materialized views over a representative workload; Chorus [6] rewrites SQL to enforce DP; DOP-SQL [7] tunes per-query mechanisms. All expose a *fixed* per-query ε_q chosen up front and account for queries in isolation; none forecasts a workload’s aggregate consumption from its repetition structure.

Workload-aware DP and caching. The matrix mechanism [10] optimizes noise for a *known* batch of linear queries, and Dong, Sun, and Yi [11] beat worst-case composition by exploiting relational structure. DP caches make repeated work cheaper at runtime: Turbo [13] conserves 1.7–15.9× budget by generalizing across overlapping linear queries, and CacheDP [14] answers a workload at lower budget under an accuracy contract. These deliver the *savings* our model prices, but reactively and without a closed-form forecast; our exact-repeat caching is a simple special case and we claim no novelty for the mechanism itself (Section 4).

Multi-query accounting and online allocation. Tight composition—moments accountant [15], Rényi DP [16], zCDP [17]—reduces the *rate* of budget spend and is orthogonal to our *count* of distinct releases. APEX [8] translates a per-query accuracy target into the minimum ε at runtime, one query at a time; BGTplanner [9] learns a Gaussian-process-plus-bandit budget policy for federated-learning rounds, driven by an observed-utility signal. Both decide budget reactively, whereas we forecast it from workload structure. A separate line *schedules* a known budget across competing tasks or analysts—privacy budget scheduling and its fairness variants, and multi-analyst budget sharing [12], which invokes the coupon-collector problem to bound queries-per-analyst (a throughput limit across *analysts*, not a distinct-template forecast). These take per-task demands as input rather than predicting them.

The model’s place. Our work sits at the intersection of two mature lines—DP-SQL accounting and unseen-species estimation—neither of which, to our knowledge, has been used to *forecast* a repeated workload’s budget. We borrow the occupancy mathematics and

Table 1: Positioning. No prior system predicts a repeated workload’s budget from its structure, before execution.

System	workload model	predict budget	exact-repeat free
PINQ/Chorus/PrivateSQL	—	no	no
Turbo / CacheDP	—	no	yes (reactive)
APEX	—	no (per-query)	no
BGTplanner	learned GP	no	no
Ours	closed-form	yes (a-priori)	yes

the exact-repeat caching mechanism (claiming novelty for neither) and contribute the link: a closed-form, before-execution prediction of $S(k)$ from $\{p_i\}$, with the limit, concentration, and utility consequences that follow.

Distinct counts and unseen species. Predicting the number of distinct templates in a length- k workload is the classical unseen-species problem. The occupancy identity $\mathbb{E}[u_k] = \sum_i [1 - (1 - p_i)^k]$ is textbook [18]; horizon-aware extrapolators—Good–Toulmin [19] and its smoothed variant [20]—predict new species from a prefix, while richness estimators such as Chao1 [21] estimate total richness. We connect these to DP-budget forecasting; query-template forecasting itself (QueryBot 5000 [22]) is non-private.

The gap. No prior DP-SQL system gives a closed-form, before-execution prediction of how much budget a *repeated* workload will consume as a function of its template-repetition structure. That prediction—and the limits, concentration, and utility consequences derived from it—is the contribution of this project.

3 Methodology

3.1 Threat model and preliminaries

A relational dataset D (DuckDB or PostgreSQL) is queried through the middleware. The adversary is any analyst who can adaptively issue supported queries and observe the noisy outputs; the goal is to bound inference about any single tuple.

DEFINITION 1 (TUPLE-LEVEL NEIGHBOURS). $D \sim D'$ iff they differ in exactly one tuple (unbounded model).

DEFINITION 2 (ε -DP [1]). $\mathcal{M}$ is ε -DP iff $\mathbb{P}[\mathcal{M}(D) \in S] \leq e^\varepsilon \mathbb{P}[\mathcal{M}(D') \in S]$ for every neighbour pair and measurable S .

Supported SQL and sensitivities. We support SELECT with COUNT/SUM/AVG, single-table FROM, conjunctive WHERE, and optional GROUP BY. Tuple-level sensitivities are $\Delta f = 1$ for COUNT, $\Delta f = B_c$ for SUM(c) with a per-column clip bound $|c| \leq B_c$, and—conservatively— $\Delta f = B_c$ for AVG (a worst-case group of size one, avoiding the implicit n -leak of the empirical-mean mechanism, since n is itself private).

Laplace release and composition. For sensitivity Δf and budget ε_q the middleware returns $\hat{f}(D) = f(D) + \eta$, $\eta \sim \text{Lap}(\Delta f / \varepsilon_q)$. A GROUP BY with g groups noises each group independently; since a tuple affects at most one group, *parallel composition* gives total cost ε_q , not $g\varepsilon_q$. This charges the aggregate *values* only, under the standard DP-histogram assumption that the group-key set is a public categorical domain fixed by the schema (e.g. the finite education levels); sensitive or unbounded key domains would require stability-based thresholding and are out of scope. Multi-aggregate queries split ε_q evenly across aggregates by sequential

composition. The exact-repeat cache returns a previously released answer with zero budget cost, since it is post-processing.

Top-k as post-processing. The middleware supports top- k (`GROUP BY g ORDER BY agg LIMIT k`) with one essential design point: ordering and truncation act on the *noised* histogram, never the true one. The engine fetches all g groups, noises every count under the same parallel composition (cost ϵ_q , not $k\epsilon_q$), and only then sorts by the noisy value and keeps the top k . The raw ORDER BY/LIMIT is stripped before it reaches the database—letting the DB apply it would select the reported groups on their *true* counts and leak which groups crossed the cutoff. Because the released slate and its counts are a deterministic function of the ϵ_q -DP noisy histogram, top- k is post-processing and charges no extra budget (a report-noisy-max guarantee covering the whole top- k list at the cost of one histogram). The price is that selection is *approximate*: groups whose true counts straddle the k -th boundary may swap. The forecast and cache apply unchanged—a repeated top- k query is one template, free on exact repeat and counted once in $\mathbb{E}[u_k]$.

Assumptions. The forecast assumes (i) a *non-adaptive* analyst whose template choices are independent of the released noise, and (ii) that the template count m and distribution $\{p_i\}$ are public properties of the *query stream*, not of the protected rows. Privacy holds regardless (composition is adaptive-safe and total spend is hard-capped at B); the assumptions concern only the average-case *forecast* (Section 6).

3.2 The analytical model

Let m be the number of distinct query templates and $\{p_i\}_{i=1}^m$ a distribution over them; the analyst issues k queries i.i.d. from $\{p_i\}$. The cache key is the exact (template, parameter) pair. Let u_k be the number of distinct templates that appear at least once.

PROPOSITION 1 (EXPECTED UNIQUE QUERIES). $\mathbb{E}[u_k] = \sum_{i=1}^m [1 - (1 - p_i)^k]$.

PROOF. Let $X_i = \mathbf{1}[\text{template } i \text{ appears}]$. Then $\mathbb{P}[X_i = 0] = (1 - p_i)^k$, so $\mathbb{E}[X_i] = 1 - (1 - p_i)^k$; linearity of expectation gives the result. $\square$

PROPOSITION 2 (BUDGET CONSUMPTION AND SAVINGS). *Naive sequential composition spends $\epsilon_{\text{naive}}(k) = k\epsilon_q$. Under workload-aware exact-repeat accounting,*

$$\mathbb{E}[\epsilon_{\text{wa}}(k)] = \epsilon_q \mathbb{E}[u_k], \quad S(k) = 1 - \frac{\mathbb{E}[\epsilon_{\text{wa}}(k)]}{\epsilon_{\text{naive}}(k)} = 1 - \frac{1}{k} \sum_{i=1}^m [1 - (1 - p_i)^k].$$

Limits. (A) *Deterministic repetition* ($p_1=1$): $S(k) = 1 - 1/k$ —the folklore rule, now a corner of Proposition 2 at maximum skew, not a standalone claim. (B) *Uniform*, $m \rightarrow \infty$: workloads almost never repeat, $\mathbb{E}[u_k] \rightarrow k$, $S(k) \rightarrow 0$, recovering naive composition. (C) *Uniform*, $k \rightarrow \infty$, m fixed: every template eventually appears, so the budget saturates at $m\epsilon_q$ regardless of k —the practical regime in which a finite-template workload cannot exhaust a budget large enough to cover all m templates.

PROPOSITION 3 (ZIPF-PARAMETERIZED WORKLOAD). *For $p_i \propto i^{-\alpha}$, the savings ratio $S(k)$ is monotone non-decreasing in α for fixed (k, m) , approaching Limit B as $\alpha \rightarrow 0$ and Limit A as $\alpha \rightarrow \infty$.*

PROOF SKETCH. $\mathbb{E}[u_k] = \sum_i \phi(p_i)$ with $\phi(x) = 1 - (1 - x)^k$ concave ($\phi'' \leq 0$), so $\mathbb{E}[u_k]$ is Schur-concave. Larger α majorizes the weight vector (mass shifts onto top templates), hence lowers $\mathbb{E}[u_k]$ and raises $S(k)$; the endpoints follow from the uniform and point-mass cases. A numerical scan over $m \in \{3, \dots, 100\}$, $k \in \{2, \dots, 200\}$ found no violation. $\square$

PROPOSITION 4 (CONCENTRATION OF u_k). *Writing $u_k = \sum_i X_i$ with $q_i = 1 - (1 - p_i)^k$, the occupancy indicators are negatively associated, so $\text{Var}[u_k] \leq \sum_i q_i(1 - q_i) \leq m/4 =: V$, and Chebyshev gives $\mathbb{P}[|u_k - \mathbb{E}[u_k]| \geq t] \leq V/t^2$.*

Because $V = O(m)$ rather than $O(k)$, the realized u_k has standard deviation $\sqrt{V} = O(\sqrt{m})$, so a *single* workload lands within a fraction of a template of the forecast (a distribution-free McDiarmid bound $2e^{-2t^2/k}$ also holds but is $\sim 18\times$ looser once u_k saturates and we report it only as a fallback). The budget-exhaustion tail follows: for $c/\epsilon_q > \mathbb{E}[u_k]$, $\mathbb{P}[u_k \epsilon_q \geq c] \leq V/(c/\epsilon_q - \mathbb{E}[u_k])^2$.

PROPOSITION 5 (UTILITY UNDER A FIXED TOTAL BUDGET). *With a fixed total ϵ_{tot} , naive affords $\epsilon_q^{\text{naive}} = \epsilon_{\text{tot}}/k$ per query and workload-aware affords $\epsilon_q^{\text{wa}} = \epsilon_{\text{tot}}/\mathbb{E}[u_k]$ per unique release; since $\mathbb{E}[|\eta|] = \Delta f/\epsilon_q$, the predicted per-query error ratio is $k/\mathbb{E}[u_k]$.*

At Zipf $\alpha=1$, $m=10$, $k=100$, $\mathbb{E}[u_k] \approx 9.9$, so workload-aware accounting answers the same workload at roughly $10\times$ lower per-query error at equal total budget. One honest caveat: a cached answer reuses a *single* noise draw, forgoing the variance reduction of k independent draws—the budget saving and the lost averaging are two sides of the same coin.

3.3 System and temporal extension

The middleware is a thin proxy between the analyst and a backend database (Figure 2). Each query is parsed and validated, hashed to a (template, parameter) key, and checked against a two-level cache; a hit is returned for free (post-processing), a miss is charged ϵ_q , noised, and cached. Four execution modes (exact, naive-DP, workload-aware-DP, temporal-DP) share the pipeline of Algorithm 1, which couples cache validity to a temporal regime: it advances a logical clock, simulates Poisson-rate invalidations, ages each entry against the staleness tolerance τ , and only then returns the cached release or charges the ledger.

A *temporal extension* couples budget to time, addressing the milestone note that “budget and temporality should be considered together”: data updates invalidate cached answers and freshness requirements force re-noising.

PROPOSITION 6 (TEMPORAL BUDGET). *With horizon T , staleness tolerance τ , update rate λ , and per-update invalidation probability q , the expected number of (re-)noisings per template is $N = \max(1, \lceil T/\tau \rceil) + T\lambda q$, so $\mathbb{E}[\epsilon_{\text{temp}}] = \epsilon_q \mathbb{E}[u_k] N$.*

Three regimes are continuous slidings of one expression: (a) *static snapshot* ($\tau \rightarrow \infty$, $\lambda=0$, $N=1$) recovers Section 3.2; (b) *bounded staleness* (τ finite, $\lambda=0$, $N=\lceil T/\tau \rceil$); (c) *update-driven* ($\lambda>0$, N grows linearly in T). The prototype is unit-tested across parser validation, sensitivity, Laplace calibration, cache semantics, the model limits, the concentration bound, and temporal re-noising.

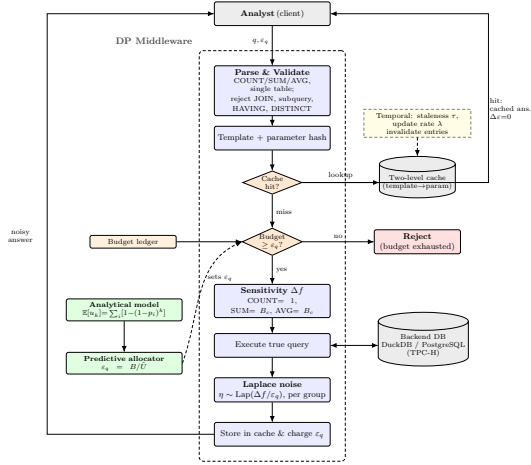


Figure 2: Workload-aware DP-SQL middleware. A cache hit is free by post-processing; a miss is charged ε_q , noised, and cached. The analytical model drives the predictive allocator ($\varepsilon_q = B/\hat{U}$), and a temporal regime (τ, λ) invalidates stale entries.

Algorithm 1 Workload-aware DP middleware (one query).

Require: Query q , budget ε_q , ledger $\mathcal{L}$, staleness τ , update (λ, q_{inv})

- 1: Parse/validate q ; advance clock; invalidate each entry w.p. λq_{inv}
- 2: Compute template hash h_T , parameter hash h_P
- 3: **if** $\mathcal{L}.\text{cache}[h_T][h_P]$ fresh (age $\leq \tau$) **then**
- 4: **return** cached $\tilde{r}$ (post-processing, $\Delta\varepsilon=0$)
- 5: **end if**
- 6: **if** $\mathcal{L}.\text{remaining} < \varepsilon_q$ **then**
- 7: **return** BUDGETEXHAUSTED
- 8: **end if**
- 9: $\Delta f \leftarrow$ sensitivity; $\mathcal{L}.\text{consumed} += \varepsilon_q$
- 10: $r \leftarrow$ execute q ; $\tilde{r} \leftarrow r + \text{Lap}(\Delta f/\varepsilon_q)$ per aggregate, per group
- 11: Cache and **return** $\tilde{r}$

4 Results and Findings

Setup. The campaign comprises **six sweeps totaling 4,155 trials** (~150,000 queries) over Zipf workloads ($m=7$ Adult age-bands and TPC-H templates), four per-query budgets $\varepsilon_q \in \{0.1, 0.5, 1, 2\}$, lengths $k \in \{10, \dots, 500\}$, and two data scales (TPC-H SF=1 with 6M lineitem rows, SF=10 with 60M). Comparisons are between internal execution modes under an identical middleware, to isolate the algorithmic effect of workload-aware accounting; end-to-end comparison against external prototypes would require query-rewriting integration and is left as future work.

4.1 Validating the model

Model vs. empirical $\mathbb{E}[u_k]$. Table 2 compares the closed form to the 30-trial empirical mean across the main grid: agreement is within $< 3\%$ in 21 of 24 cells; the three outliers (3.5–8.6%) occur at

Table 2: Model vs. empirical u_k , Zipf ($m=7$, 30 trials/cell). Parentheses: signed relative error (%).

$\alpha \backslash k$	10	25	50	100
0.0	5.50/5.53 (−0.6)	6.85/6.77 (1.2)	7.00/7.00 (0.0)	7.00/7.00 (0.0)
0.5	5.30/5.23 (1.3)	6.73/6.73 (−0.1)	6.98/6.93 (0.7)	7.00/7.00 (0.0)
1.0	4.73/4.70 (0.6)	6.32/6.30 (0.3)	6.88/6.90 (−0.3)	7.00/7.00 (0.0)
1.5	3.98/3.87 (2.8)	5.57/5.70 (−2.3)	6.48/6.40 (1.3)	6.91/6.93 (−0.3)
2.0	3.25/3.13 (3.5)	4.64/4.70 (−1.3)	5.69/5.73 (−0.7)	6.50/6.43 (1.1)
3.0	2.19/2.07 (5.7)	3.07/3.33 (−8.6)	3.85/3.90 (−1.3)	4.72/4.67 (1.1)

high skew and small k , where $\mathbb{E}[u_k]$ is tiny and a 0.1-unit deviation is a large relative error. The savings ratio $S(k)$, computed from measured budget, matches within $< 2\%$ in 23/24 cells.

Invariance and limits. The model’s $\mathbb{E}[u_k]$ is structurally independent of ε_q (confirmed within 0.12 units across $\varepsilon_q \in \{0.1, 0.5, 1, 2\}$) and of data scale: the same templates over a 10× larger TPC-H database (60M rows) give identical empirical $\mathbb{E}[u_k]$ to within 0.034 units, isolating workload structure from data scale. Pushing skew to $\alpha=10$ at $k=200$, predicted and empirical $\mathbb{E}[u_k]$ stay within 0.13 units while $\mathbb{E}[u_k]$ varies by an order of magnitude, and $S(k) \rightarrow 99.4\%$ recovers the $1 - 1/k$ corner numerically (Figure 3).

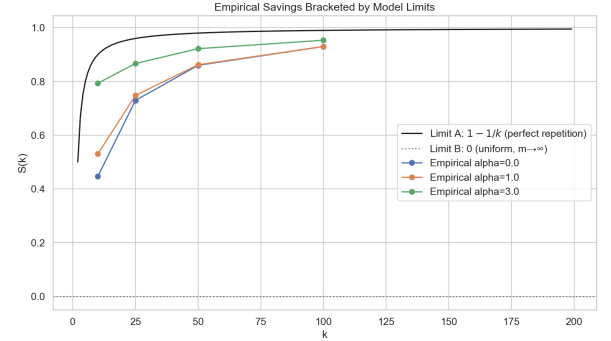


Figure 3: Savings ratio $S(k)$ vs. workload length, several Zipf α . Dashed: model. Markers: empirical mean (30 trials). High- α curves approach $1 - 1/k$ (Limit A); the $\alpha=0$ curve stays near 0 until k approaches m (Limit B).

Robustness. Four properties make the forecast dependable. (i) *Distribution-agnostic*: it is exact for any i.i.d. marginal (uniform, Zipf, lognormal-shaped), matching the empirical mean to ≤ 0.2 . (ii) *Concentrated*: by Proposition 4, $\sqrt{V}=0.34\text{--}1.22$ at $m=20$, so a single run is reliable, not merely the expectation. (iii) *Exact on heterogeneous workloads*: on a constructed mixed W1+W4 workload with true $u_k = 1 + (1-f)k$, the closed form $\varepsilon_q u_k$ matches the measured budget to the cent at every mixing fraction (Table 3). (iv) *Conservative under correlation*: a bursty sticky-Markov process (same stationary marginal) lowers the distinct count, so the i.i.d. forecast over-predicts u_k by $1.09/1.29/2.72\times$ at stickiness $s=0.3/0.6/0.9$ —the safe direction, since $\varepsilon_q \mathbb{E}[u_k]$ then only loosens as an upper bound and the privacy guarantee is never violated. The worst case of this guarantee is the

$\varepsilon_q=B/m$ allocator: since $u_k \leq m$ for *any* arrival process, it never rejects.

Table 3: Mixed W1+W4 workload, $B=50$, $\varepsilon_q=0.5$, 20 trials. The closed form matches the measured budget exactly.

f	true u_k	measured ε used	predicted $\varepsilon_q u_k$
0.1	91	45.5	45.5
0.3	71	35.5	35.5
0.5	51	25.5	25.5
0.7	31	15.5	15.5
0.9	11	5.5	5.5

Temporal extension. The temporal model is validated by sweeping τ (with $\lambda=0$) and λ (with τ large): the prediction $\mathbb{E}[\varepsilon_{\text{temp}}] = \varepsilon_q \mathbb{E}[u_k]N$ tracks the empirical mean in trend and matches closely at large τ (rare re-noising), while at small τ it is a conservative *upper* bound (e.g. at $\tau=10$ it predicts $\mathbb{E}[\varepsilon] \approx 70$ vs. an empirical ≈ 39 , since not all re-noisings materialize within the horizon). Freshness thus has an explicit, bounded cost.

4.2 Real-workload validation (Redbench)

The synthetic sweep cannot establish *external* validity, so we add the first test on production-derived traces: Redbench [23], 30 analytical SQL workloads synthesized from Amazon Redshift’s Redset query log, in which each user’s real query timeline (and thus its repetition structure) is mapped onto CEB+/JOB query instances. The exact-query identity is the SQL instance, so the realized distinct count u_k and Redbench’s own *query-repetition rate* $= 1 - u_k/k$ are exactly our $S(k)$. Running the occupancy model against all 30 workloads (experiments/redbench_validation.py) yields four findings. (1) *Real workloads span the full skew range*: realized $S(k)$ runs from 0% to 98% (mean 53%), so template repetition—the structure the model exploits—is pervasive in production, not an artefact of the sweep. (2) *Real distributions are heavy-tailed*: fitting Zipf to each workload gives $\alpha \in [0, 3]$ with mean 0.84 (near classic Zipf), and our synthetic sweep $\alpha \in \{0, \dots, 3\}$ brackets the real range—answering “why Zipf?” with production data. (3) *The closed form tracks realized savings* to within a mean absolute 0.13 in $S(k)$, tightest in the high-repetition regime where savings are largest (e.g. 0.92 vs. 0.93, 0.97 vs. 0.98). (4) *Forecasting from a 50% prefix*, the smoothed Good–Toulmin estimator [20] halves the plug-in’s error (0.12 vs. 0.22 in u_k/k), confirming on real data that unseen-species correction is the right tool for prefix-based forecasting. Honestly, Redbench is trace-derived synthesis—its repetition structure is real but its queries are CEB+/JOB instances, not raw production SQL—so this raises external validity substantially without fully eliminating it (Section 6).

4.3 Budget savings and where naive composition fails

Full grid. Across six workloads, workload-aware accounting answers the same 100 queries at comparable per-query MAE while spending only $\varepsilon_q u_k$: savings of 100 $\times$ on the repetitive W1, 33 $\times$ /20 $\times$ /14.3 $\times$ on the TPC-H/uniform pools (where u_k saturates at the pool size m),

and *exactly* 1 $\times$ on the genuine drill-down W4, where every query is unique and the model correctly predicts $S(k)=0$ (Figure 4). These multipliers are accounting ratios $1/(1 - S(k))$ that any exact-repeat cache attains; our contribution is the closed-form *prediction* of $S(k)$ before execution (within $< 3\%$), not the saving itself.

Table 4: Full grid (6 workloads, $k=100$, $\varepsilon_q=1$, 30 trials). Workload-aware spends $\varepsilon_q u_k$ at 93–99% cache-hit rates; the drill-down (W4) is the predicted no-win case.

Workload	naive ε	work.-aware ε	savings
W1 Repetitive	100.0	1.0	100 $\times$
W2 TPC-H returnflag	100.0	3.0	33 $\times$
W2 TPC-H priority	100.0	5.0	20 $\times$
W2 Zipf $\alpha=1$	100.0	7.0	14.3 $\times$
W3 Uniform	100.0	7.0	14.3 $\times$
W4 Drill-down (unique)	100.0	100.0	1 $\times$

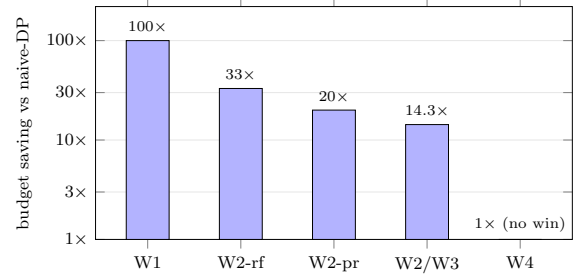


Figure 4: Budget saved vs. naive ($k=100$, $\varepsilon_q=1$, log scale): up to 100 $\times$ on the repetitive W1 and 14–33 $\times$ on the parametric/uniform pools, but exactly 1 $\times$ (no win) on the unique drill-down W4—the model predicts the $S(k)=0$ case rather than overclaiming.

Real-system baseline. To confirm these savings against a *deployed* engine rather than our own naive mode, we run the same workloads through OpenDP SmartNoise-SQL [24] (experiments/real_system_baseline.py). SmartNoise charges every query against its odometer and has no exact-repeat caching, so on the repetitive dashboard ($k=100$ identical COUNT releases at $\varepsilon_q=1$) it spends the full $\varepsilon=100.0$ where workload-aware accounting spends 1.0 ($u_k=1$; the 99 repeats are free)—a 100 $\times$ gap, now against a real library, not an internal baseline. On a genuinely all-distinct workload both spend 100.0: we *tie*, since there is no repetition to exploit. Both use basic pure- ε composition here; SmartNoise’s tighter Rényi accounting would lower *both* rates and is orthogonal to (composes with) the caching—consistent with the model treating tight composition as a separate axis.

Two concrete failure regimes of naive composition. On the real middleware (Adult, $B=10$, $k=100$, 12 trials; Table 5), repetition turns the structural advantage into a stark gap. (A) *Budget exhaustion*: a dashboard re-issuing one tile at fixed $\varepsilon_q=1$ exhausts naive after $B/\varepsilon_q=10$ queries (10/100 answered), while caching answers all 100/100 for free—a 10 $\times$ gap. (B) *Accuracy at fixed total budget*: to

answer all 100, naive must thin the budget ($\epsilon_q=B/k=0.1$, error ≈ 10), whereas concentrating it on the $u_k=1$ distinct release makes the answer essentially exact. (*C, honest counter*): on a genuine drill-down both match at error ≈ 9.6 —included precisely because the model predicts the no-win case.

Table 5: Naive’s failure regimes vs. workload-aware (real middleware, Adult, $B=10$, $k=100$).

Workload	Naive ans.	err	Workload-aware ans.	err
Repetitive, fixed $\epsilon_q=1$	10/100	0.9	100/100	0.4
Repetitive, fixed budget B	100/100	10.0	100/100	≈ 0
Drill-down (all unique)	100/100	9.6	100/100	9.6

4.4 Putting the model to work: predictive allocation

The model is descriptive (it predicts budget from a workload) and also *prescriptive*: an allocator can estimate $\hat{U} = \mathbb{E}[u_k]$ online from the empirical template distribution (after a short warmup) and set $\epsilon_q = B/\hat{U}$ to target the offline optimum $\epsilon_q^* = B/u_k$ (Proposition 2), capping spend at B . At a fixed *total* budget this trades coverage for accuracy: on Zipf workloads ($k=100$, $B=10$) it cuts per-query MAE versus naive (Table 6), by point estimates of 16.8% at $\alpha=0$ down to 5.4% at $\alpha=2$. We are deliberately careful: a paired t -test finds the gain significant only at low skew ($p \approx 0.05$ at $\alpha=0$; $p > 0.3$ for $\alpha \geq 1$), so it is a low-skew effect, not a uniform win, and it costs some late-query rejections. The robust, regime-independent result is instead the *safe* allocator $\epsilon_q=B/m$, which never rejects ($u_k \leq m$) and matches the best operating point of a learned bandit at zero exploration cost (Appendix A). The MAE reduction is largest in the tightest-budget regime, where every release matters: sweeping $B \in \{1, 5, 10, 50\}$ at $\alpha=1$, the predictive allocator cuts MAE by 32% at $B=1$ (predictive 68.7 vs. naive 101.1) and 21% at $B=50$. The advantage is structural, not data-dependent: at TPC-H SF=10 (60M rows) it still cuts MAE to 6.03 vs. naive’s 10.01 (40%).

Table 6: Predictive allocator vs. fixed- ϵ_q baselines (mean $\pm$ SE, 30 trials, $k=100$, $B=10$). The gain over naive is significant only at $\alpha=0$.

α	Mode	MAE	answered	ϵ spent
0.0	Naive	9.86	100/100	10.0
	Predictive	8.21$\pm$0.85	69/100	10.0
1.0	Naive	9.81	100/100	10.0
	Predictive	8.98$\pm$0.93	78/100	10.0
2.0	Naive	9.96	100/100	10.0
	Predictive	9.42$\pm$1.40	88/100	10.0

4.5 Privacy leakage

We complement the budget/utility analysis with two attacks that confirm the per-query calibration. *Membership inference*: an adversary observing one noisy COUNT must decide whether a target is present. Empirical AUC tracks the DP upper bound $e^\epsilon/(1+e^\epsilon)$

within $\leq 1\%$ and is no better than chance at $\epsilon \leq 0.1$ (Table 7, left). *Reconstruction*: replaying a 5-level differencing drill-down on real Adult counts (48842, 34327, 24137, 7210, 3137), the mean absolute error scales as $1.5/\epsilon$ (the expected difference of two $\text{Lap}(1/\epsilon)$ variables) to within a few percent across three orders of magnitude (Table 7, right). A stronger shadow-model MIA across four workloads $\times$ four ϵ_q (32 cells, 60 shadow runs each) yields AUC 0.14–0.58, far below the cumulative-budget bound’s $\rightarrow 1.0$: *cumulative-budget bounds are loose for realistic workload MIA*, and the per-query ϵ_q , not the cumulative budget, governs the signal. Both attacks degrade exactly where the model predicts efficient budgeting (high skew, low ϵ_q), and both succeed on the W4 drill-down—which the model also flags as the no-savings case.

Table 7: Single-release leakage. Left: membership-inference AUC vs. the DP bound. Right: differencing reconstruction error vs. the $1.5/\epsilon$ reference.

ϵ	AUC	$\frac{e^\epsilon}{1+e^\epsilon}$	ϵ	rec. err	$1.5/\epsilon$
0.01	0.499	0.503	0.01	148.2	150.0
0.10	0.522	0.525	0.10	14.8	15.0
1.00	0.724	0.731	1.00	1.5	1.5
5.00	0.988	0.993	5.00	0.3	0.3

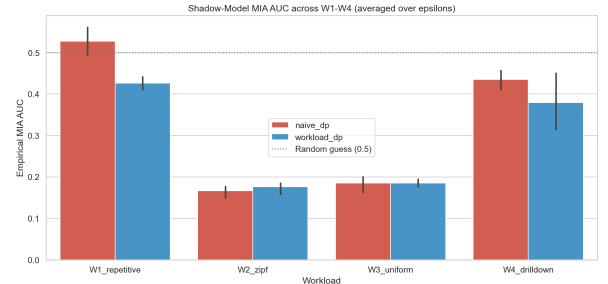


Figure 5: Shadow-model MIA AUC across W1–W4 at $\epsilon_q \in \{0.1, 0.5, 1, 2\}$ (32 cells, 60 shadow runs each). Even a classifier seeing the full $k=50$ sequence stays far below the cumulative-budget bound ($\rightarrow 1.0$); most queries do not directly probe the target tuple.

4.6 Runtime and overhead

The privacy machinery is cheap relative to I/O. Instrumenting 1,500 queries, cache hits are $2.7\times$ faster than misses (0.76 vs. 2.02 ms) because they skip the database round-trip (36% of miss cost); SQL parsing is 21%, and the privacy-specific stages (sensitivity, allocation, noise) sum to under 0.25 ms. Tail latency is controlled (miss $p99=5.1$ ms, hit $p99=3.6$ ms), and single-threaded throughput is ~ 495 q/s on the miss path and $\sim 1,320$ q/s on the hit path, so a workload-aware mode at 93–99% hit rate sustains analyst-tier load. The asymptotic cost of the forecast itself is $O(m)$ per evaluation of $\mathbb{E}[u_k]$.

5 Future Work

Several extensions follow directly. (1) *Raw production traces*: the Redbench validation uses trace-derived synthesis; running the model on raw analyst query logs (and executing them through a DP engine end-to-end) would close the remaining external-validity gap. (2) *Tight composition*: replacing basic sequential composition with Rényi DP [16] or zCDP [17] would predict $\sum \varepsilon_i$ rather than count u_k , with the closed-form structure mostly unchanged—and composes with the caching, as the SmartNoise comparison shows. (3) *Multi-table / JOINs*: the occupancy framework generalizes to JOIN-shaped queries given a per-template sensitivity bound such as residual sensitivity [26] or R2T [27]. (4) *Equivalence-checked semantic cache*: pairing tree-kernel/embedding similarity with a symbolic equivalence prover (predicate normalization, range merging, α -conversion) would let the cache safely admit non-identical-but-equivalent queries (Appendix B). (5) *Cache re-release and AVG decomposition*: re-noising stale cache entries on budget surplus, and a SUM+COUNT decomposition of AVG with per-group accounting, would tighten utility. (6) *Private group-key domains*: a stability-based threshold that suppresses low-count groups would lift the public-key-domain assumption at a small δ cost.

6 Discussion and Conclusion

Findings. (1) Budget consumption is a function of $\mathbb{E}[u_k]$, not of the cache implementation: $\mathbb{E}[\varepsilon_{wa}] = \varepsilon_q \mathbb{E}[u_k]$ holds within $< 3\%$ in 21/24 cells, exactly on the mixed workload, and is invariant to scale and ε_q —so a deployment can estimate its privacy cost *before* running a query. (2) The $1 - 1/k$ rule is the high-skew corner of one model. (3) Cumulative- ε bounds are loose for realistic MIA (AUC 0.14–0.58). (4) The model is honest about failure: on uniform workloads and drill-downs, savings vanish and reconstruction succeeds, exactly as predicted.

Practical implication. Because $\mathbb{E}[u_k]$ is computable before any query runs and is invariant to data scale and ε_q , the custodian of the dashboard scenario can size ε_q *a priori*: the safe choice $\varepsilon_q = B/m$ provably finishes the month (since $u_k \leq m$), and the predictive allocator pushes accuracy further when the workload is skewed. The model also tells the custodian when caching cannot help—a drill-down workload spends like naive composition—so the privacy/utility budget can be planned, not discovered after exhaustion. The same forecast bounds the leakage analysis: budgeting and attack-resistance improve together in the high-skew, low- ε_q regime.

Threats to validity. We separate three. *Construct*: the headline “ $\mathbb{E}[u_k]$ within $< 3\%$ ” is a Monte-Carlo *self-consistency* check—the empirical u_k is sampled from the very Zipf distribution the closed form integrates, so the mean converges to $\mathbb{E}[u_k]$ *by construction*; it validates the estimator’s arithmetic, not the claim that real workloads are i.i.d. Zipf. *Internal*: the forecast assumes a non-adaptive analyst; against a worst-case adaptive analyst the only valid bound is $u_k \leq \min(k, m)$, i.e. the $\varepsilon_q = B/m$ safe allocator. *External*: the headline grid is synthetic, but we now validate on Redbench (Section 4.2)—30 production-derived traces spanning 0–98% repetition

with fitted Zipf $\alpha \in [0, 3]$ (mean 0.84)—which raises external validity substantially. The residual gap is honest: Redbench is trace-derived synthesis (real repetition structure, benchmark queries rather than raw production SQL), and the i.i.d. closed form is mildly over-optimistic about savings at the low-repetition end. Scope is single-table aggregates under basic composition; JOINs and tight (Rényi/zCDP) composition are future work.

Conclusion and reproducibility. A closed-form model parameterized by $(\{p_i\}, k)$ predicts realized DP-budget consumption in repeated aggregate SQL workloads, recovers the folklore $1 - 1/k$ rule as a corner, and is validated to $< 3\%$ across 4,155 trials—while being explicit about the regimes in which it predicts no benefit. The middleware, all experiment generators, and the canonical result CSVs are public¹ with fixed seeds; the model-validation and allocation experiments are bit-reproducible and the large campaign is distributionally reproducible. The Redbench and SmartNoise-SQL baselines are reproduced by experiments/redbench_validation.py and experiments/realsystem_baseline.py.

A Predictive Budget Allocator

The model is also *prescriptive*: instead of fixing ε_q offline, an allocator can estimate $\hat{U} = \mathbb{E}[u_k]$ online from the empirical (Laplace-smoothed) template distribution after a short warmup and set $\varepsilon_q = B/\max(\hat{U}, 1)$, capped to the remaining budget so total spend never exceeds B . This targets the offline optimum $\varepsilon_q^* = B/u_k$ (Proposition 2). On Zipf workloads ($k=100$, $B=10$, 30 trials) the predictive allocator cuts per-query MAE by up to $\sim 17\%$ at low skew (point estimates 16.8% at $\alpha=0$ down to 5.4% at $\alpha=2$, and up to 32% in the tightest-budget regime $B=1$), but the gain is statistically significant only at low skew (paired t -test $p \approx 0.05$ at $\alpha=0$, $p > 0.3$ for $\alpha \geq 1$); we report it as a low-skew effect, not a uniform win. The regime-independent result is instead the *safe* allocator $\varepsilon_q = B/m$: since $u_k \leq m$, it never rejects and needs no warmup, and on the benchmark it answers all queries at fresh-release MAE 2.0 versus an ε -greedy bandit’s 3.8 at the same 100% rate—a $1.9\times$ gap that is the bandit’s exploration tax, since B/m is the bandit’s best-arm operating point (holding when $m \leq k/4$).

Table 8: Online allocation policies (Zipf, $m=20$, $k=100$, $B=10$, 60 trials). Fresh MAE (over cache misses) is reported jointly with the answered fraction.

Policy	Fresh MAE	Answered	ε used
Naive ($\varepsilon_q = B/k$)	10.1	100/100	1.6
ε -greedy bandit	3.8	100/100	6.1
Safe closed form (B/m)	2.0	100/100	8.0
Oracle (B/u_k , needs forecast)	1.6	100/100	10.0

The allocator’s quality hinges on predicting u_k from a warmup prefix. The default plug-in occupancy estimate uses only observed templates and so under-predicts; horizon-aware extrapolators do better in the realistic regime where the warmup is at least a third of the workload ($t = (k-n)/n \leq 2$): the Good-Toulmin family roughly halves the error, and the smoothed variant [20] tames its divergence past the $t \leq 1$ validity limit (Table 9). Chao1 [21]

¹<https://github.com/canoztas/cmp653-project>

estimates total richness, not the horizon- k count, and over-predicts. Better u_k accuracy is a coverage/noise trade-off, not a free win: smoothed-GT lifts the answered fraction from 81% to 92% at higher per-release noise.

Table 9: u_k -prediction MAE (%) by estimator and extrapolation factor t (Zipf, $m=50$, $k=100$). Lower is better.

estimator	$t=0.5$	$t=1$	$t=2$	$t=4$
Plug-in occupancy	18	30	43	58
Good-Toulmin [19]	8	17	193	204
Smoothed GT [20]	8	17	27	151
Chao1 [21]	55	48	47	47

B A Negative Result: Semantic Caching

We tested whether structurally similar queries the exact cache misses could be matched via a Collins–Duffy tree kernel [25] plus an AST sentence-embedding (cosine), admitting a match only when both scores exceed conservative thresholds. The semantic cache fails in *both* directions: it admits false positives (queries differing only in WHERE literals have near-identical ASTs but different true answers, so a reused cached value is wrong by thousands of count units), and it misses textbook equivalences (commutative AND reordering, $\text{age} \geq 30$ vs. $\text{age} > 29$), because ordered-child kernels and text embeddings do not encode logical equivalence. Returning a cached answer for a non-equivalent query also falls *outside* the post-processing guarantee. The lesson: tree-kernel/embedding similarity is not equivalence; a safe semantic cache needs a symbolic equivalence prover (predicate normalization, range merging, α -conversion), which we leave as future work.

C Glossary

Template / u_k : a query with WHERE literals placeholdered; u_k is the number of distinct templates in a length- k workload. **Occupancy model**: $\mathbb{E}[u_k] = \sum_i [1 - (1 - p_i)^k]$, the expected number of non-empty cells when k balls fall into m cells with probabilities $\{p_i\}$. **Savings $S(k)$** : fractional budget saved versus naive composition, $1 - \mathbb{E}[u_k]/k$. **Parallel composition**: a GROUP BY costs ε_q , not $g\varepsilon_q$, because each tuple affects one group. **Post-processing**: any function of a DP output is DP at no extra cost (cache hits and top- k selection). **Safe allocator B/m** : per-query budget that never rejects because $u_k \leq m$.

References

- [1] C. Dwork, F. McSherry, K. Nissim, A. Smith. Calibrating Noise to Sensitivity in Private Data Analysis. *TCC*, 2006.
- [2] C. Dwork, A. Roth. The Algorithmic Foundations of Differential Privacy. *Found. Trends TCS*, 2014.
- [3] I. Dinur, K. Nissim. Revealing Information While Preserving Privacy. *PODS*, 2003.
- [4] F. McSherry. Privacy Integrated Queries (PINQ). *SIGMOD*, 2009.
- [5] I. Kotsogiannis et al. PrivateSQL: A Differentially Private SQL Query Engine. *PVLDB* 12(11), 2019.
- [6] N. Johnson, J. P. Near, D. Song. Towards Practical Differential Privacy for SQL Queries (Chorus). *PVLDB* 11(5), 2018.
- [7] Y. Yu et al. DOP-SQL: Differentially Private Optimization for SQL. 2024.
- [8] C. Ge, X. He, I. F. Ilyas, A. Machanavajjhala. APEx: Accuracy-Aware Differentially Private Data Exploration. *SIGMOD*, 2019.
- [9] X. Zhang et al. BGTplanner: Maximizing Training Accuracy for Differentially Private Federated Recommenders via Strategic Privacy Budget Allocation. *IEEE Trans. Services Computing* 18(6), 2025, 3523–3531.
- [10] C. Li, G. Miklau, M. Hay, A. McGregor, V. Rastogi. The Matrix Mechanism: Optimizing Linear Counting Queries under Differential Privacy. *Vldb Journal* 24(6), 2015.
- [11] W. Dong, J. Sun, K. Yi. Better than Composition: How to Answer Multiple Relational Queries under Differential Privacy. *SIGMOD*, 2023.
- [12] D. Pujol, A. Sun, B. Fain, A. Machanavajjhala. Multi-Analyst Differential Privacy for Online Query Answering. *PVLDB* 16(4), 2023.
- [13] K. Kostopoulou, P. Tholoniati, A. Cidon, R. Geambasu, M. Lécuyer. Turbo: Effective Caching in Differentially-Private Databases. *SOSP*, 2023.
- [14] M. Mazmudar, T. Humphries, et al. Cache Me If You Can: Accuracy-Aware Inference Engine for Differentially Private Data Exploration. *PVLDB* 16(4), 2023.
- [15] M. Abadi et al. Deep Learning with Differential Privacy. *CCS*, 2016.
- [16] I. Mironov. Rényi Differential Privacy. *CSF*, 2017.
- [17] M. Bun, T. Steinke. Concentrated Differential Privacy. *TCC*, 2016.
- [18] P. Flajolet, D. Gardy, L. Thimonier. Birthday Paradox, Coupon Collectors, Caching Algorithms and Self-Organizing Search. *Discrete Applied Math.* 39(3), 1992.
- [19] I. J. Good, G. H. Toulmin. The Number of New Species, and the Increase in Population Coverage, when a Sample is Increased. *Biometrika* 43, 1956.
- [20] A. Orlitsky, A. T. Suresh, Y. Wu. Optimal Prediction of the Number of Unseen Species. *PNAS* 113(47), 2016.
- [21] A. Chao. Nonparametric Estimation of the Number of Classes in a Population. *Scand. J. Statistics* 11(4), 1984.
- [22] L. Ma et al. Query-based Workload Forecasting for Self-Driving Database Management Systems (QueryBot 5000). *SIGMOD*, 2018.
- [23] A. Renen et al. Redbench: A Benchmark Reflecting Real Workloads (synthesized from Amazon Redshift’s Redset traces). *aiDM @ SIGMOD*, 2025. github.com/utndatasystems/redbench.
- [24] OpenDP Project. SmartNoise-SQL: Differentially Private SQL Queries. github.com/opendp/smartnoise-sdk.
- [25] M. Collins, N. Duffy. Convolution Kernels for Natural Language. *NeurIPS*, 2001.
- [26] W. Dong, K. Yi. Residual Sensitivity for Differentially Private Multi-Way Joins. *SIGMOD*, 2021.
- [27] W. Dong, J. Fang, K. Yi, et al. R2T: Instance-optimal Truncation for Differentially Private Query Evaluation with Foreign Keys. *SIGMOD*, 2022.